

Neural Networks and Deep Learning

Perceptrons, activations, backpropagation, gradient descent, and taming overfitting

1. From Neurons to Networks

A neural network is a model family loosely inspired by the brain: it is built from many small computing units, called neurons or units, wired together in layers. Each unit forms a weighted sum of its inputs, adds a bias term, and passes the result through a nonlinear *activation function*. Stacking layers of such units lets the network build progressively more abstract representations — edges from pixels, shapes from edges, objects from shapes.

The adjective *deep* in deep learning simply means the network has many layers. Depth is what gives modern networks their power: each layer transforms its input into a slightly more useful representation, so that by the final layer even a simple linear classifier can separate classes that were hopelessly tangled in the raw input space.

2. The Perceptron

The perceptron, introduced in 1958, is the simplest neural unit. It computes a weighted sum of its inputs and outputs 1 if the sum exceeds a threshold, else 0. A single perceptron can learn any *linearly separable* pattern — for example, the logical AND function — by adjusting its weights whenever it makes a mistake.

Its famous limitation is the XOR problem: no single perceptron can represent exclusive-or, because the two classes are not separable by one straight line. The historical fix was to stack perceptrons into multiple layers, which leads directly to the multilayer networks of the next sections.

3. Activation Functions

Without nonlinearity, a stack of layers collapses mathematically into a single linear transformation, no matter how deep it is. Activation functions supply the needed nonlinearity.

- **Sigmoid:** squashes any input into $(0, 1)$. Once the default choice; now rare in hidden layers because its gradients vanish for large inputs.
- **Tanh:** squashes into $(-1, 1)$; zero-centered, which often trains faster than sigmoid, but still suffers vanishing gradients.
- **ReLU (rectified linear unit):** outputs $\max(0, x)$. Cheap to compute and largely immune to vanishing gradients, making it the modern default for hidden layers.
- **Softmax:** turns a vector of raw scores into a probability distribution over classes; the standard final-layer activation for classification.

Choosing activations is one of the small decisions that compound: ReLU in hidden layers plus softmax (classification) or linear (regression) outputs is a reliable starting template.

4. Multilayer Networks and Deep Learning

A multilayer perceptron (MLP) chains layers of units: an input layer that receives the features, one or more hidden layers that transform them, and an output layer that produces predictions. With enough hidden units, even a single-hidden-layer network can approximate any continuous function — the *universal approximation theorem*. In practice, depth (depth = many layers) generalizes better than width for most real problems, because hierarchical features mirror the compositional structure of images, language, and sound.

Training such networks was considered impractical for decades, until three developments converged: large labeled datasets, graphics processors that parallelize the matrix multiplications at the heart of the computation, and algorithmic refinements — better initializations, ReLU activations, and adaptive optimizers — that made deep training stable.

5. Training Networks with Gradient Descent

Neural networks are trained exactly like other models: define a loss function, then minimize it with gradient descent. For classification the standard loss is *cross-entropy*, which heavily penalizes confident wrong answers; for regression it is usually mean squared error.

The practical challenge is scale. A modern network can have hundreds of millions of parameters, and computing the gradient of the loss with respect to every one of them by naive means would be impossibly slow. The field therefore relies on *mini-batch* gradient descent — estimating the gradient on a few dozen or hundred examples at a time — combined with adaptive learning-rate methods such as Adam, which give each parameter its own effective step size. The missing piece is an efficient way to obtain all those gradients in the first place, which is precisely what backpropagation provides.

6. Backpropagation

Backpropagation (backprop) is the algorithm that computes the gradient of the loss with respect to every weight in the network, efficiently, in a single backward pass. It is not an optimizer itself — it is the gradient-computation engine that *feeds* gradient descent.

Here is the relationship in plain terms. Training needs to answer, for each weight, the question: if I nudge this weight slightly, how does the loss change? The chain rule of calculus lets us propagate that question backward: the loss depends on the output layer, which depends on the previous layer, and so on back to the first weights. Backpropagation applies the chain rule layer by layer, from outputs toward inputs, reusing intermediate results so the total cost is comparable to a single forward pass rather than one pass per weight.

So: **backpropagation computes the gradients; gradient descent takes the steps**. Every training iteration is a duet — a forward pass to compute predictions and loss, a backward pass (backprop) to compute gradients, and a gradient-descent update that nudges each weight downhill. Understanding this division of labor demystifies most of deep learning's training machinery.

7. Overfitting in Deep Learning

Deep networks, with their enormous capacity, overfit enthusiastically: given enough parameters, a network can memorize its entire training set, noise included, while generalizing poorly. The warning signs are the familiar ones — training loss keeps falling while validation loss stalls or rises — and the remedies are the same ideas introduced in *Introduction to Machine Learning*, plus a few deep-learning specialties.

- **More data:** still the most reliable cure; augmentation (rotations, crops, paraphrases) manufactures it cheaply.
- **Dropout:** randomly deactivates a fraction of units during each training step, preventing fragile co-adaptations.
- **Weight decay (L2 regularization):** penalizes large weights, keeping the function smooth.
- **Early stopping:** monitors validation loss and halts training at its minimum, before memorization sets in.
- **Batch normalization:** stabilizes layer inputs, which incidentally has a mild regularizing effect.

As with classical models, the way to *detect* overfitting is the honest train/validation/test discipline plus learning curves — techniques covered in depth in the companion document *Machine Learning Evaluation*.

8. Convolutional and Recurrent Networks

Two architectural families deserve special mention. *Convolutional neural networks* (CNNs) use sliding filters that share weights across spatial positions, making them translation-invariant and dramatically more parameter-efficient for images. *Recurrent neural networks* (RNNs) feed their own previous output back as input, giving them a form of memory suited to sequences; modern practice has largely replaced them with the *Transformer*, whose attention mechanism processes entire sequences in parallel and underpins today's large language models.

Whatever the architecture, the training story is unchanged: forward pass, backpropagated gradients, gradient-descent updates, and vigilant evaluation against overfitting.